Université Mohammed Premier
Faculté des Sciences, Oujda
Département d'Informatique
Responsable : E.M. DAOUDI

Année universitaire 2021-2022
Filière SMI
Module : Programmation Système
Semestre : S6

Examen de la Session Ordinaire. Durée : 1h30
Juin 2022

Exercice 1 (12 pts)

1. Donnez l'arbre de processus générés par le programme C ci-dessous :

```
main() {
        fork() || fork();
}
```

2. Donnez l'arbre de processus générés par le programme C ci-dessous :

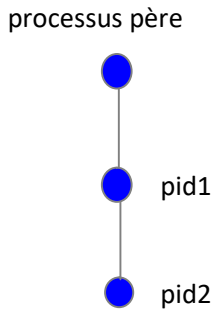
```
main() {
    if (fork() || fork())
        fork() || fork();
}
```

3. Donnez l'arbre de processus générés par le programme ci-dessous :

```
main() {
    if (fork() || fork())
        if (fork() || fork())
            fork() ;
}
```

4. Soient « f() », « g() », « f1() », « f2() » et « g2() » cinq fonctions données. Considérons l'arbre de processus ci-dessous. On suppose que :

- le processus père exécute la fonction « f() » suivie de la fonction « g() »,
- le processus « pid1 », fils du processus père, exécute la fonction « f1() »,
- le processus « pid2 », fils du processus « pid1 », exécute la fonction « f2() » suivie de la fonction « g2() ».



4.1. Ecrire un programme C qui permet de générer l'arbre ci-dessous où chaque processus exécute son code, de plus chaque processus doit attendre la terminaison de son fils avant de se terminer.

4.2. L'objectif de cette question est de synchroniser l'exécution des processus père et « pid2 ». Modifier le programme C de la question 4.1 afin que :

- le processus père ne peut commencer l'exécution de la fonction « g() » qu'après l'exécution de la fonction « f() » (exécutée par le processus père) et de de la fonction « f2() » (exécutée par le processus « pid2 »).

N.B.

- **Ne pas utiliser les tubes pour communiquer les identifiants des processus.**
- **Ne pas utiliser la fonction « sleep() » pour synchroniser.**

Exercice 2 (8 pts)

Le but de cet exercice est de synchroniser l'accès aux variables partagées « v », « x », « y » et « z », de type « int », initialisées à 0, par plusieurs threads en utilisant les **sémaphores d'exclusion mutuelle (les Mutexs)**. Soient « f() », « g1() », « g2() » et « g3 () » quatre fonctions données.

1. Dans cette question, on suppose que le thread principal **« th0 »** crée les threads **« th1 » et « th2 »** et exécute le code « Code0 » alors que le thread « th1 » exécute le code « Code1 » et le thread « th2 » exécute le code « Code2 ».

Code0 :	**Code1**:	**Code2**:

while (1){
v=f(y);
}
	while (1){
	x=g1(v);
	}
		while (1){
		y=g2(x);
		}

Ecrire un programme C (ou donner une solution théorique claire) qui permet de synchroniser, à chaque itération, les modifications des variables « v », « x » et « y » en respectant les contraintes de synchronisation suivantes :

A chaque itération:

- Le thread principal modifie la variable « v ».
- Après modification de la variable « v » par le thread « th0 », le thread « th1 » modifie la variable « x ».
- Après modification de la variable « x » par le thread « th1 », le thread « th2 » modifie la variable « y ».
- Le thread principal, ne peut recommencer la modification de la variable « v », à l'itération suivante, qu'après modification de la variable « y » par le thread « th2 ».

2. Dans cette question, on suppose que le thread principal **« th0 »** crée les threads **« th1 », « th2 » et « th3 »** et exécute le code « Code0 » alors que le thread « th1 » exécute le code « Code1 », le thread « th2 » exécute le code « Code2 » et le thread « th3 » exécute le code « Code3 ».

Code0 :	**Code1**:	**Code2**:	**Code3**:

while (1){
v=f(x, z);
}
	while (1){
	x=g1(v);
	}
		while (1){
		y=g2(v);
		}
			while (1){
			z=g3(y)
			}

Ecrire un programme C (ou donner une solution théorique claire) qui permet de synchroniser, à chaque itération, les modifications des variables « v », « x », « y » et « z » en respectant les contraintes de synchronisation suivantes :

A chaque itération:

- Le thread principal modifie la variable « v ».
- Après modification de la variable « v » par le thread « th0 », les threads « th1 » et « th2 » modifient respectivement les variables « x » et « y ».
- Après modification de la variable « y » par le thread « th2 », le thread « th3 » modifie la variable « z ».
- Les modifications des variables « x » et « y » sont complètement indépendantes et ne s'effectuent pas dans un ordre précis.
- Le thread principal, ne peut recommencer la modification de la variable « v », à l'itération suivante, qu'après modification des variables « x » et « z » par les threads « th1 » et « th3 ».

3. Modifier le programme C (ou la solution théorique) de la question précédente en supposant que, pendant la première itération, on commence par modifier les variables « x » et « y ».

Université Mohammed Premier
Faculté des Sciences, Oujda
Département d'Informatique
Responsable : E.M. DAOUDI

Année universitaire 2021-2022
Filière SMI
Module : Programmation Système
Semestre : S6

Correction de l'Examen de la Session Ordinaire
Juin 2022

Exercice 1 (12pts)

1. Donnez l'arbre de processus générés par le programme ci-dessous.

```
fork() || fork()
```

2. Donnez l'arbre de processus générés par le programme ci-dessous.

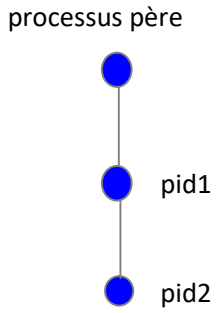
```
main() {
    if (fork() || fork())
        fork() || fork();
    }
```

3. Donnez l'arbre de processus générés par le programme ci-dessous.

```
main() {
    if (fork() || fork())
        if (fork() || fork())
            fork() ;
}
```

4. Considérons l'arbre de processus ci-dessous. On suppose que :

- le processus père exécute la fonction « f() » suivie de la fonction « g() »
- le processus « pid1 » exécute la fonction « f1() ».
- le processus « pid2 » exécute la fonction « f2() » suivie de la fonction « g2() ».



4.1. Ecrire un programme C dans qui permet de générer l'arbre ci-dessous. Chaque processus fils exécute son code. Chaque processus doit attendre la terminaison de son fils avant de se terminer.

4.2. L'objectif de cette question est de synchroniser l'exécution des processus père et « pid2 ». Modifier le programme de la question 1 afin que :

- le processus « pid2 » ne peut commencer l'exécution de la fonction « g2() » qu'après l'exécution des fonction « f2() » (exécutée par « pid2 ») et « f() » (exécuté par le processus père).
- de même, le processus père ne peut commencer l'exécution de la fonction « g() » qu'après l'exécution des fonction « f2() » (exécutée par « pid2 ») et « f() » (exécuté par le processus père).

N.B.

- **Ne pas utiliser les tubes pour échanger les identifiants des processus.**
- **Ne pas utiliser la fonction « sleep() » pour synchroniser.**

Remarques :

- Pour bien comprendre, on introduit des variables dans lesquelles on sauvegarde les valeurs retournées par les appels des « fork() ».
- L'instruction « a || b » est vrai (différents de 0) si « a » est vrai ou « b » est vrai. Donc on n'évalue « b » que si « a » est faux.

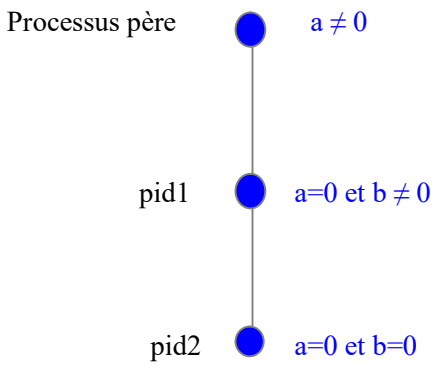
Reponses :

Question 1

Evaluation de l'expression (a=fork()) || (b=fork())

- Tout d'abord la première instruction (a=fork()) est exécutée par le processus père qui crée un premier fils nommé « pid1 » :
  - Dans le processus père, la valeur retournée par « fork() » est différente de 0, donc ($a \neq 0$).
  - Dans le processus fils « pid1 », la valeur retournée par « fork() » est égale à 0, donc ($a = 0$).
- La deuxième instruction (b=fork()) n'est exécutée que si la valeur retournée par le premier « fork() » est fausse (égale à 0) c'est à dire si « a =0 », donc le deuxième « fork() » est exécutée par le processus fils « pid1 » qui crée un fils nommé « pid2 ».
  - Dans le processus « pid1 » on a : (a=0) et ($b \neq 0$)
  - Dans le processus fils « pid2 » (clone de « pid1 ») on a : (a=0) et (b=0).

Donc l'arbre de processus générés par l'évaluation de l'expression (a=fork()) || (b=fork()) est le suivant :



Question 2

Evaluation de l'instruction

```
if ((a=fork()) || (b=fork()))
        (c=fork()) || (d=fork());
}
```

L'instruction (c=fork()) || (d=fork()) n'est évaluée que si l'expression (a =fork()) || (b=fork()) est vrai (c'est-à-dire si ($a \neq 0$) ou ($b \neq 0$)).

L'expression (a =fork()) || (b=fork()) est exécutée par les processus père, « pid1 » et « pid2 ». Donc d'après la question 1, on a

- ($a \neq 0$) dans le processus père, donc cette expression retourne vrai dans le processus père.
- ($b \neq 0$) dans le processus « pid1 », donc cette expression retourne vrai dans le processus « pid1 ».
- (a=0) et (b=0) dans le processus « pid2 », donc cette expression est fausse dans le processus « pid2 ».

par conséquent l'instruction (c=fork()) || (d=fork()); est exécutée par les processus père et « pid1 ».

Evaluation de l'expression (c=fork()) || (d=fork() par le processus père

D'après la première question on a:

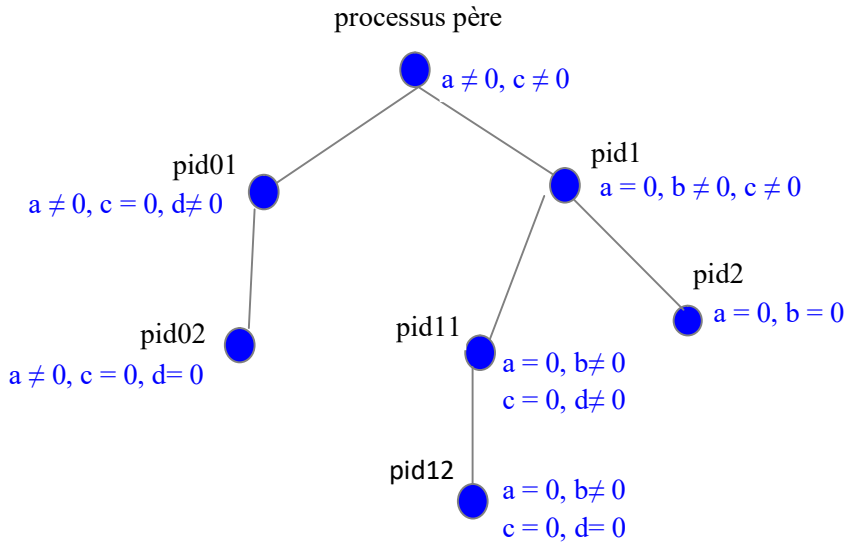
- Le processus père crée un deuxième fils nommé « pid01 » et la valeur retournée par « c=fork() » est différente de 0, donc (c ≠ 0) dans le processus père.
- La deuxième instruction (d=fork()) est exécutée par le processus fils « pid01 » qui crée un fils nommé « pid02 ».
    - Dans le fils « pid01 » on a : (c=0) et (d ≠ 0)
    - Dans le processus fils « pid02 » (clone de « pid01 ») on a : (c=0) et (d=0).

Evaluation de l'expression (c=fork()) || (d=fork() par le processus « pid1 »

D'après la première question on a:

- Le processus « pid1 » crée un deuxième fils nommé « pid11 » et la valeur retournée par « c=fork() » est différente de 0, donc (c ≠ 0) dans « pid1 ».
- La deuxième instruction (d=fork()) est exécutée par le processus fils « pid11 » qui crée un fils nommé « pid12 ».
    - Dans le fils « pid11 » on a : (c=0) et (d ≠ 0)
    - Dans le processus fils « pid12 » (clone de « pid11 ») on a : (c=0) et (d=0).

Donc l'arbre de processus générés est le suivant :



Question 3

Evaluation de l'instruction

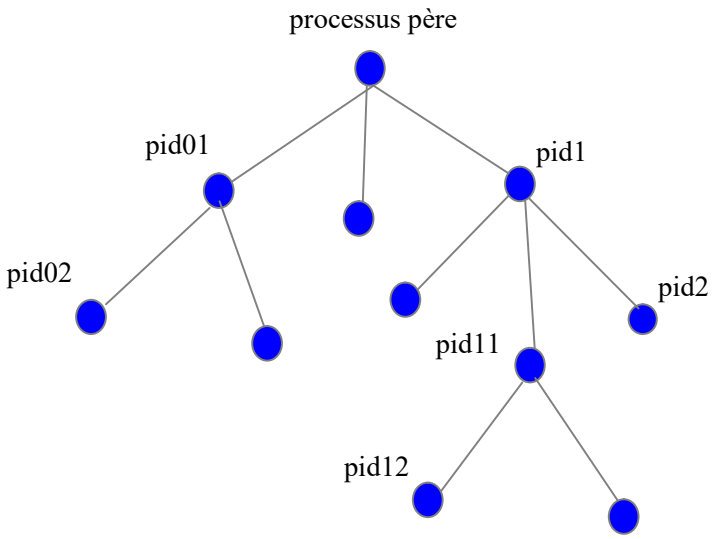
```
if ((a=fork()) || (b=fork()))
        if ((c=fork()) || (d=fork()))
            e=fork() ;
}
```

L'instruction « e=fork() » n'est évaluée que si l'expression ((c=fork()) || (d=fork())) est vrai (c'est à-dire si (c ≠ 0) ou (d ≠ 0)). D'après ce qui précède, on a :

- (c ≠ 0) dans les processus père et « pid1 » donc cette expression est vrai dans les processus père et « pid1 ».
- (d ≠ 0) dans les processus « pid01 » et « pid11 », donc cette expression est vrai dans les processus « pid01 » et « pid11 ».

Donc l'instruction « e=fork() » est exécutée par les processus père, « pid1 », « pid01 » et « pid11 » qui créent chacun un processus fils.

Donc l'arbre de processus générés est le suivant.



Réponses question 4

Question 4.1.

Code C

```
…
main(){
    pid_t   pid1, pid2;
    pid1=fork();   // le processus père crée le processus fils « pid1 »
    if (pid1==0) { // si je suis le processus « pid1 »
        pid2=fork();   // le processus « pid1 » crée le processus fils « pid2 »
        if (pid2==0) { // si je suis le processus « pid2 »
            f2() ;   // exécute la fonction f2()
            g2() ;   // exécute la fonction g2()
            }
        }
        else { // si je suis le processus « pid1 »
            f1() ;   // exécute la fonction f1()
            wait(0) ;   // attend la terminaison de son fils
            }
        }
    else {   // si je suis le processus père
            f();   // exécute la fonction f()
            g();   // exécute la fonction g()
            wait(0);   // attend la terminaison de son fils
            }
    }
}
```

Question 4.2.

1ère solution

Dans cette solution,

- le processus « pid2 » envoie un signal au processus père après exécution de la fonction f2().
- Le processus père, après exécution de la fonction « f() », attend le signal envoyé par « pid2 », ensuite exécute la fonction « g() ».

Principe de la solution :

Processus pid2	**Processus pid1**	**Processus père**
- Exécute f2(); - Avertit le processus père qu'il a exécuté f2() en lui envoyant un signal. Donc « pid2 » doit connaitre le pid du processus père. - Exécute la fonction g2().	- Crée le processus « pid2 ». - Exécute la fonction f1(). - Attend la terminaison de son fils « pid2 ».	- Crée le processus « pid1 ». - Exécute la fonction f(); - attend le signal envoyé par « pid2 ». - Après réception du signal, il exécute la fonction g(). - Attend la terminaison de son fils « pid1 ».

Remarque importante :

Pourque le « pid » du processus père soit connu dans le processus « pid2 », le père doit sauvegarder son « pid » dans une variable, par exemple dans la variable « pid_pere » avant la création du processus « pid1 ». Puisque « pid2 » et un clone du processus « pid1 » qui est un clone du processus père, la valeur de « pid_père » sera connue dans le processus « pid2 ».

Code C complet

```
Définition des handlers
void fonc_hand (int sig) { };
main(){
    pid_t    pid1, pid2, pid père ;
    pid_pere=getpid() ; // le processus père sauvegarde son pid dans la variable « pid_pere ».
    pid1=fork();    // le processus père crée le fils « pid1 »
    if (pid1==0) { // si je suis le processus « pid1 »
        pid2=fork();    // le processus « pid1 » crée le fils « pid2 »
        if (pid2==0) { // si je suis le processus « pid2 »
            f2() ;    // « pid2 » exécute la fonction f2()
            kill(pid_pere,SIGUSR2) ; // « pid2 » envoie le signal « SIGUSR2 » au processus père
            g2() ; // « pid2 » exécute la fonction g2()
        }
        else { // si je suis le processus « pid1 »
            f1() ; // « pid1 » exécute la fonction f1()
            wait(0) ;    // « pid1 » attend la terminaison de son fils « pid2 »
        }
    }
    else { // si je suis le processus père
        // installe un nouveau handler « fonc_hand() » pour le signal «SIGUSR2»
        // on peut remplacer « fonc_hand() » par « SIG_IGN » pour ignorer le signal «SIGUSR2»
        signal(SIGUSR2, fonc_hand);
        f();    // le processus père exécute la fonction f()
        pause();    // attend l'arrivé du signal pour continuer l'exécution de la fonction g()
        g();    // après reception du signal, le processus père exécute la fonction g()
        wait(0); // le processus père attend la terminaison de son fils « pid1 »
    }
}
```

2ème Solution

Dans la 1ère solution, il y a risque de blocage. En effet, si le signal arrive au processus père avant «l'instruction « pause() », le signal sera traité (exécution de la fonction handler) avant de faire pause. Une fois arrivé à l'instruction « pause() ; », le processus père reste bloqué sur l'attente d'un signal. Pour éviter le problème de blocage, on remplace l'instruction « pause() » par une attente active après exécution de la fonction « f() ». On introduit une variable « v » initialisée à 0. Sa valeur sera modifiée après réception du signal.

Code C complet

```
Définition des handlers
int v=0 ;
void fonc_hand (int sig) {
    v=1;
};
main(){
    pid_t   pid1, pid2, pid père ;
    pid_pere=getpid() ; // le processus père sauvegarde son pid dans la variable « pid_pere ».
    pid1=fork();   // le processus père crée le fils « pid1 »
    if (pid1==0) {   // si je suis le processus « pid1 »
        pid2=fork();   // le processus « pid1 » crée le fils « pid2 »
        if (pid2==0) { // si je suis le processus « pid2 »
            f2() ;   // « pid2 » exécute la fonction f2()
            kill(pid_pere,SIGUSR2) ;   // «pid2» envoie le signal «SIGUSR2» au processus père
            g2() ;   // « pid2 » exécute la fonction g2()
        }
        else { // si je suis le processus « pid1 »
            f1() ;   // « pid1 » exécute la fonction f1()
            wait(0) ;   // « pid1 » attend la terminaison de son fils « pid2 »
        }
    }
    else { // si je suis le processus père
        // installe un nouveau handler « fonc_hand() » pour le signal «SIGUSR2»
        signal(SIGUSR2, fonc_hand);
        f(); // le processus père exécute la fonction f()
        pause(); // attend l'arrivé du signal pour continuer l'exécution de la function g()
        while (v==0); // attente active.
        g();   // après réception du signal, le processus père exécute la fonction g()
        wait(0);   // le processus père attend la terminaison de son fils « pid1 »
    }
}
```

3ème solution

Dans cette solution :

- le processus « pid2 », après exécution de la fonction « f2() », envoie un signal à son père (le processus « pid1 ») et continue l'exécution de la fonction « g2() ».
- le processus « pid1 », après réception du signal envoyé par « pid2 », envoie un signal au processus père.
- Le processus père, après exécution (ou en cours d'exécution) de la fonction « f() », attend le signal envoyé par « pid1 ». Après réception du signal, il exécute la fonction « g() ».

Principe de la solution :

Processus pid2	**Processus pid1**	**Processus père**
- Exécute f2(); - Avertit le processus « pid1 » qu'il a exécuté f2() en lui envoyant le signal « SIGUSR2 ». - Exécute la fonction g2().	- Crée le processus « pid2 ». - Exécute la fonction f1(). - Après réception du signal « SIGUSR2 », envoie le signal « SIGUSR1 » au processus père. - Attend la terminaison de son fils « pid2 ».	- Crée le processus « pid1 ». - Exécute la fonction f(); - attend le signal « SIGUSR1 » envoyé par « pid1 ». - Après réception du signal, il exécute la fonction g(). - Attend la terminaison de son fils « pid1 ».

Code C complet

```
Définition des handlers
void fonc_hand(int sig) { };
void fonc_hand1 (int sig) {
    kill(getppid(),SIGUSR1) ; // envoie le signal « SIGUSR1 » au processus père
};

main(){
    pid_t   pid1, pid2;
    pid1=fork();   // le processus père crée le fils « pid1 »
    if (pid1==0) {   // si je suis le processus « pid1 »
        pid2=fork();   // le processus « pid1 » crée le fils « pid2 »
        if (pid2==0) {   // si je suis le processus « pid2 »
            f2() ;   // « pid2 » exécute la fonction f2()
            kill(getppid(), SIGUSR2) ; // « pid2 » envoie le signal « SIGUSR2 » au processus
                                       // « pid1 » (le père de « pid2 »)
            g2() ; // « pid2 » exécute la fonction g2()
        }
        else { // si je suis le processus « pid1 »
            // installe un nouveau handler « fonc_hand1() » pour le signal «SIGUSR2»
            // à la réception du signal «SIGUSR2», il exécute la fonction « fonc_hand1() » qui
            // consiste à envoyer le signal « SIGUSR1 » au processus père.
            signal(SIGUSR2, fonc_hand1);
            f1() ; // « pid1 » exécute la fonction f1()
            wait(0) ;   // « pid1 » attend la terminaison de son fils « pid2 »
        }
    }
    else { // si je suis le processus père
        // installe un nouveau handler « fonc_hand() » pour le signal «SIGUSR1»
        // on peut remplacer «fonc_hand()» par « SIG_IGN » pour ignorer le signal «SIGUSR1»
        signal(SIGUSR1, fonc_hand);
        f();   // le processus père exécute la fonction f()
        pause();   // attend l'arrivé du signal pour continuer l'exécution de la function g()
        g();   // après reception du signal, le processus père exécute la fonction g()
        wait(0); // le processus père attend la terminaison de son fils « pid1 »
    }
}
```

4ème solution

Comme dans la 3ème solution, on peut remplacer "pause()" par une attente active pour éviter le blocage qui peut être généré par l'arrivée du signal avant de faire "pause()".

Exercice 2 (8 pts)

Le but de cet exercice est de synchroniser l'accès aux variables partagées « v », « x », « y » et « z », de type « int », initialisées à 0, par plusieurs threads en utilisant les **sémaphores d'exclusion mutuelle (les Mutexs)**. Soient « f() », « g1() », « g2() » et « g3 () » quatre fonctions données.

1. Dans cette question, on suppose que le thread principal **« th0 »** crée les threads **« th1 » et « th2 »** et exécute le code « Code0 » alors que le thread « th1 » exécute le code « Code1 » et le thread « th2 » exécute le code « Code2 ».

Code0 :	**Code1**:	**Code2**:

while (1){
v=f(y);
}
	while (1){
	x=g1(v);
	}
		while (1){
		y=g2(x);
		}

Ecrire un programme C (ou donner une solution théorique claire) qui permet de synchroniser, à chaque itération, les modifications des variables « v », « x » et « y » en respectant les contraintes de synchronisation suivantes :

A chaque itération:

- Le thread principal modifie la variable « v ».
- Après modification de la variable « v » par le thread « th0 », le thread « th1 » modifie la variable « x ».
- Après modification de la variable « x » par le thread « th1 », le thread « th2 » modifie la variable « y ».
- Le thread principal, ne peut recommencer la modification de la variable « v », à l'itération suivante, qu'après modification de la variable « y » par le thread « th2 ».

2. Dans cette question, on suppose que le thread principal **« th0 »** crée les threads **« th1 », « th2 » et « th3 »** et exécute le code « Code0 » alors que le thread « th1 » exécute le code « Code1 », le thread « th2 » exécute le code « Code2 » et le thread « th3 » exécute le code « Code3 ».

Code0 :	**Code1**:	**Code2**:	**Code3**:

while (1){
v=f(z);
}
	while (1){
	x=g1(v);
	}
		while (1){
		y=g2(v);
		}
			while (1){
			z=g3(x, y);
			}

Ecrire un programme C (ou donner une solution théorique claire) qui permet de synchroniser, à chaque itération, les modifications des variables « v », « x », « y » et « z » en respectant les contraintes de synchronisation suivantes :

A chaque itération:

- Le thread principal modifie la variable « v ».
- Après modification de la variable « v » par le thread « th0 », les threads « th1 » et « th2 » modifient respectivement les variables « x », « y ».
- Après modification des variables « x » et « y » par les threads « th1 » et « th2 », le thread « th3 » modifie la variable « z ».
- Les modifications des variables « x » et « y » sont complètement indépendantes et ne s'effectuent pas dans un ordre précis.
- Le thread principal, ne peut recommencer la modification de la variable « v », à l'itération suivante, qu'après modification de la variable « z » par les threads « th3 ».

3. Dans cette question, on suppose que le thread principal **« th0 »** crée les threads **« th1 », « th2 » et « th3 »** et exécute le code « Code0 » alors que le thread « th1 » exécute le code « Code1 », le thread « th2 » exécute le code « Code2 » et le thread « th3 » exécute le code « Code3 ».

Code0 :	**Code1**:	**Code2**:	**Code3**:

while (1){
v=f(x, z);
}
	while (1){
	x=g1(v);
	}
		while (1){
		y=g2(v);
		}
			while (1){
			z=g3(y)
			}

Ecrire un programme C (ou donner une solution théorique claire) qui permet de synchroniser, à chaque itération, les modifications des variables « v », « x », « y » et « z » en respectant les contraintes de synchronisation suivantes :

A chaque itération:

- Le thread principal modifie la variable « v ».
- Après modification de la variable « v » par le thread « th0 », les threads « th1 » et « th2 » modifient respectivement les variables « x », « y ».
- Après modification de la variable « y » par le thread « th2 », le thread « th3 » modifie la variable « z ».
- Les modifications des variables « x » et « y » sont complètement indépendantes et ne s'effectuent pas dans un ordre précis.
- Le thread principal, ne peut recommencer la modification de la variable « v », à l'itération suivante, qu'après modification des variables « x » et « z » par les threads « th1 » et « th3 ».

4. Modifier le programme (la solution théorique) de la question précédente en supposant que, pendant la première itération, on commence par modifier les variables « x » et « y ».

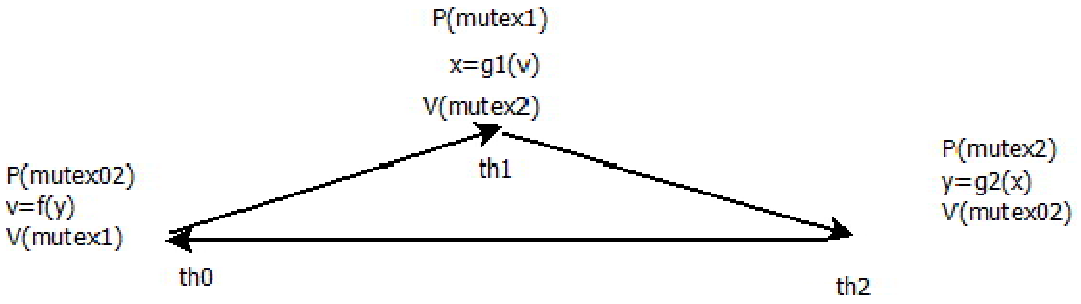
Réponses

Question 1:

On synchronise à chaque itération les threads: le thread principal « th0 » et les threads « th1 » et « th2 » :

Remarque :

- Le graphe de dépendance d'exécution des threads est le suivant :
- Pour que le thread principal « th0 » commence le premier, il doit verrouiller les mutexs : « mutex1 » et « mutex2 » avant la création des threads « th1 » et « th2 ».



Solution théorique

Thread principal th0	Thread th1	Thread th2

- Verrouiller les mutexs « mutex1 » pour garantir que le thread principal commence avant le threads th1.
- Verrouiller en plus le mutex « mutex2 » pour garantir que le thread th1 commence avant le thread th2.
- Créer les thread th1 et th2.
- Exécuter les instructions
while (1){
P(mutex02)
v=f(y) ;
V(mutex 1)
}
	- Exécuter les instructions
	while (1) {
	P(mutex1)
	x=g1(v) ;
	V(mutex2)
	}
		- Exécuter les instructions
		while (1) {
		P(mutex2)
		y=g2(x) ;
		V(mutex02)
		}

Programme en langage C :

```
#include <stdio.h>
#include <pthread.h>
int v=0, x=0, y=0;
pthread_t   t h1, th2;
pthread_mutex_t   mutex1, mutex2, mutex02;
void *g1 (void* k);    //   déclaration de la fonction thread1
void *g2 (void* k);    //   déclaration de la fonction thread2

main ( ) {
  // initialisation des mutex
  pthread_mutex_init (&mutex1,NULL);
  pthread_mutex_init (&mutex2,NULL);
  pthread_mutex_init (&mutex02,NULL);

// Verrouiller les mutex : mutex1 et mutex2
  pthread_mutex_lock(&mutex1);
  pthread_mutex_lock(&mutex2);

  // création des threads
  pthread_create (&th1, NULL, g1, NULL);
  pthread_create (&th2, NULL, g2, NULL);

  while (1){
      pthread_mutex_lock(&mutex02);
      v=f(y);
      pthread_mutex_unlock(&mutex1);
  }
}
```

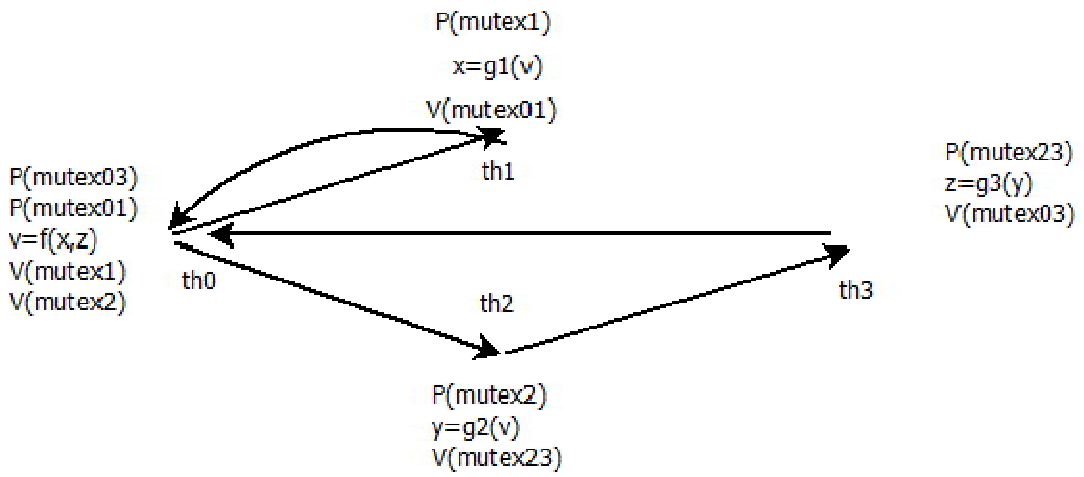
```
void *g1 (void* k) { // exécuté par le thread th1
  while (1) {
      pthread_mutex_lock(&mutex1);
      x=g1(v);
      pthread_mutex_unlock(&mutex2);
  }
  pthread_exit(NULL);
}

void *g2 (void* k) { // exécuté par le thread th2
  while (1) {
      pthread_mutex_lock(&mutex2);
      y=g2(x);
      pthread_mutex_unlock(&mutex02);
  }
  pthread_exit(NULL);
}
```

Question 2:

On synchronise à chaque iteration les threads: le thread principal « th0 », lees threads « th1 », « th2 » et « th3 »:

Remarque :

- Le graphe de dépendance d'exécution des threads est ci-dessous :
- Pour que le thread principal « th0 » commence le premier, il doit verrouiller les mutexs : « mutex1 », « mutex2 » et « mutex23 » avant la création des threads « th1 », « th2 » et « th3 ».



Solution théorique

Thread principal th0	Thread th1	Thread th2	Thread th3

- Verrouiller les mutexs « mutex1 » et « mutex2 » pour garantir que le thread principal commence avant les threads th1 et th2.
- Verrouiller en plus le mutex « mutex23 » pour garantir que le threads th2 commencent avant le thread th3.
- Créer les thread th1, th2 et th3
- Exécuter les instructions
while (1){
P(mutex03)
P(mutex01)
v=f(x,z) ;
V(mutex 1)
V(mutex 2)
}
	- Exécuter les instructions
	while (1) {
	P(mutex1)
	x=g1(v) ;
	V(mutex01)
	}
		- Exécuter les instructions
		while (1) {
		P(mutex2)
		y=g2(v) ;
		V(mutex23)
		}
			- Exécuter les instructions
			while (1) {
			P(mutex23)
			z= g3(y) ;
			V(mutex03)
			}

Programme en langage C :

```
#include <stdio.h>
#include <pthread.h>
int v=0, x=0, y=0, z=0;
pthread_t   t h1, th2, th3;
pthread_mutex_t   mutex1, mutex2, mutex23, mutex01, mutex03;
void *g1 (void* k);    //   déclaration de la fonction thread1
void *g2 (void* k);    //   déclaration de la fonction thread2
void *g3 (void* k);    //   déclaration de la fonction thread3

main ( ) {
   // initialisation des mutex
   pthread_mutex_init (&mutex1,NULL);
   pthread_mutex_init (&mutex2,NULL);
   pthread_mutex_init (&mutex23,NULL);
   pthread_mutex_init (&mutex01,NULL);
   pthread_mutex_init (&mutex03,NULL);

// Verrouiller les mutex : mutex1, mutex2 et mutex23
   pthread_mutex_lock(&mutex1);
   pthread_mutex_lock(&mutex2);
   pthread_mutex_lock(&mutex23);
```

```
  // création des threads
  pthread_create (&th1, NULL, g1, NULL);
  pthread_create (&th2, NULL, g2, NULL);
  pthread_create (&th3, NULL, g3, NULL);

  while (1){
      pthread_mutex_lock(&mutex01);
      pthread_mutex_lock(&mutex03);
      v=f(x,z);
      pthread_mutex_unlock(&mutex1);
      pthread_mutex_unlock(&mutex2);
  }
}
void *g1 (void* k) { // exécuté par le thread th1
  while (1) {
      pthread_mutex_lock(&mutex1);
      x=g1(v);
      pthread_mutex_unlock(&mutex01);
  }
  pthread_exit(NULL);
}
void *g2 (void* k) { // exécuté par le thread th2
  while (1) {
      pthread_mutex_lock(&mutex2);
      y=g2(v);
      pthread_mutex_unlock(&mutex23);
  }
  pthread_exit(NULL);
}
void *g3 (void* k) { // exécuté par le thread th2
  while (1) {
      pthread_mutex_lock(&mutex23);
      z=g3(y);
      pthread_mutex_unlock(&mutex03);
  }
  pthread_exit(NULL);
}
```

Question 3:

Pour s'assurer que les thread1 « th1 » et « th2 » commencent les premiers avant le thread « th3 » et que « th3 » commence avant le thread principal, le thread principal doit

- verrouiller les mutexs « mutex03 » et « mutex01 » pour que le thread « th0 » doit attend la libération des mutex « mutex01 » et « mutex03 » par les threads « th1 » et « th3 » (thread principal attend les threads « th1 » et « th3 »).
- verrouiller en plus le mutex « mutex23 » avant la création du thread « th3 » pour s'assurer que le thread « th3 » doit attendre la libération du mutex « mutex23 » par le thread « th2 ».